



**UNIVERSIDAD
DON BOSCO**

Semana 7

Los 4 Pilares de la POO



Herencia · Polimorfismo · Encapsulamiento · Abstracción

Herencia

Polimorfismo

Encapsulamiento

Abstracción

¿Qué lograrás al terminar esta clase?

01

Implementas herencia con ':'

Reutilizas código de una clase base en clases hijas.

02

Aplicas polimorfismo con virtual/override

Un mismo método actúa diferente según el objeto real.

03

Usas modificadores de acceso correctamente

public, private, protected — quién puede ver qué.

04

Diseñas clases abstract e interfaces

Contratos que obligan a las subclases a implementar comportamiento.

05

Reconoces los 4 pilares en código real

Combinas los 4 pilares para diseñar jerarquías limpias.

Contenido de la Clase

Semana 7 — Los 4 Pilares de la POO

01

Herencia

Reutilizar y extender clases

02

Polimorfismo

Un método, muchas formas

03

Encapsulamiento

Proteger el estado interno

04

Abstracción

Clases abstract e interfaces

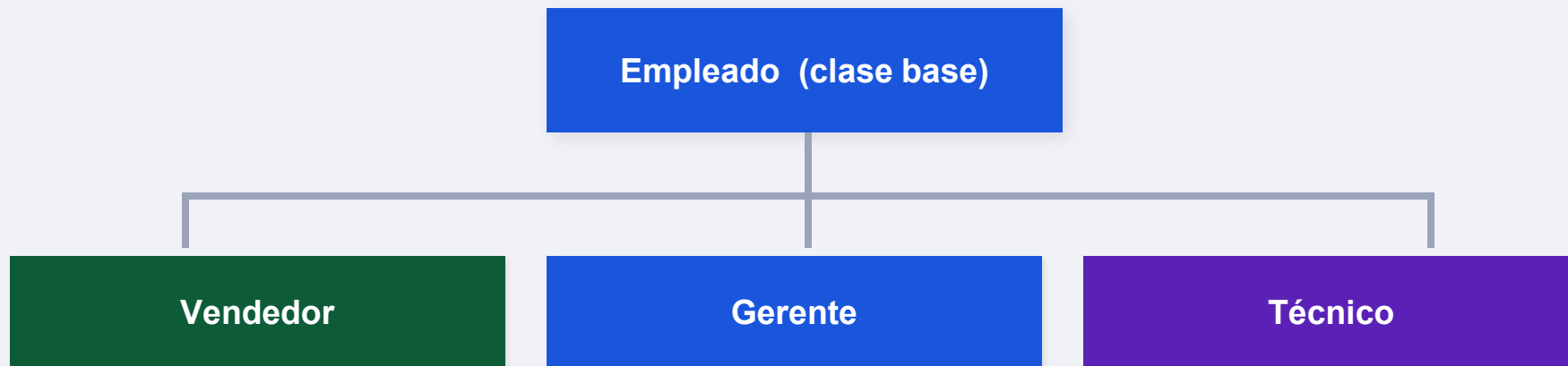
01

Herencia

Reutilizar y extender código de forma elegante

¿Qué es la herencia? — el concepto

💡 La herencia permite que una clase HIJA herede propiedades y métodos de una clase PADRE, sin tener que reescribirlos. La hija puede agregar sus propias cosas o modificar el comportamiento del padre.



*Hereda: Nombre, Cargo, SalarioBase,
CalcularSueldo(), ToString()*

🔑 Regla: usa herencia cuando la relación es IS-A (ES-UN). Vendedor ES-UN Empleado ✅. No abuses — 'Moto ES-UN Vehículo' sí; 'Motor ES-UN Vehículo' no.

Sintaxis completa — clase base y clases hijas

```
// — CLASE BASE
```

```
class Empleado
{
    public string Nombre    { get; set; }
    public string Cargo     { get; set; }
    public double SalarioBase { get; protected set; }
} // ← protected: hijas lo ven
```

```
    public Empleado(string nombre, string cargo,
double salario)
    {
        Nombre      = nombre;
        Cargo       = cargo;
        SalarioBase = salario;
    }
```

```
    public virtual double CalcularSueldo() // ←
```

```
// Uso en Program.cs
```

```
var e = new Empleado(
    "Ana", "Dev", 1200);
```

```
var g = new Gerente(
    "Luis", 1500, 500);
```

```
e.MostrarInfo();
// Dev: Ana | $1,200.00
```

```
g.MostrarInfo();
// Gerente: Luis | $2,000.00
```

```
// Gerente HEREDA MostrarInfo
// pero su CalcularSueldo
// devuelve salario + bono
```

Palabras clave: protected · base · sealed

protected

Visible dentro de la clase Y en todas sus clases hijas.
Invisible desde afuera.

```
class Empleado {  
    protected double SalarioBase;  
}  
class Gerente : Empleado {  
    void Test() => SalarioBase *= 1.1; // ✓  
}
```

base

Referencia a la clase padre. Llama su constructor o sus métodos.

```
class Gerente : Empleado {  
    public Gerente(string n, double s)  
        : base(n, "Gerente", s) { }  
  
    public override string ToString()  
        => base.ToString() + $" Bono:{Bono:C}";  
}
```

sealed

Evita que una clase sea heredada (o que un método sea sobreescrito).

```
sealed class DirectorGeneral : Gerente {  
    // Nadie puede heredar de aquí  
}  
// class CEO : DirectorGeneral { } // ✗ Error
```


02

Polimorfismo

Un método, muchas formas

POLIMORFISMO

¿Qué es el polimorfismo? — el concepto

💡 Poli = muchos, morfo = formas. El polimorfismo permite que objetos de distintas clases respondan al MISMO mensaje de forma diferente. Una lista de Empleado puede tener Gerentes y Vendedores — cada uno calcula su sueldo a su manera.

`emp.CalcularSueldo()`

↓ *el mismo mensaje*

Empleado

→ \$1,200 (solo base)

Gerente

→ \$2,000 (base+bono)

Vendedor

→ \$1,700 (base+comisión)

🔑 El polimorfismo funciona gracias a virtual + override. .NET decide en TIEMPO DE EJECUCIÓN cuál versión llamar, según el tipo real del objeto (late binding).

virtual · override · lista polimórfica

```
class Empleado {  
    public virtual double CalcularSueldo() =>  
    SalarioBase;  
}  
  
class Gerente : Empleado {  
    public override double CalcularSueldo() =>  
    SalarioBase + Bono;  
}  
  
class Vendedor : Empleado {  
    public double Comision { get; set; }  
    public override double CalcularSueldo() =>  
    SalarioBase + Comision;  
}  
  
// — LISTA POLIMÓRFICA  
  
var plantilla = new List<Empleado> // tipo base
```

¿Por qué funciona?

La variable es List<Empleado>

Pero los objetos son Gerente y Vendedor

.NET mira el tipo REAL en runtime

Llama el override correcto

¡Sin if/else por tipo!

03

Encapsulamiento

Proteger el estado interno del objeto

¿Qué es el encapsulamiento? — el concepto

💡 Encapsular = envolver los datos y protegerlos. Los datos internos del objeto son `private` y solo se accede a ellos por métodos o propiedades controladas. Así evitas que alguien ponga un saldo negativo directamente.

Modificador	¿Quién accede?	Cuándo usarlo
<code>public</code>	Todos — dentro y fuera de la clase, cualquier ensamblado	Métodos y propiedades que forman la interfaz pública
<code>private</code>	Solo dentro de la misma clase	Campos, implementación interna
<code>protected</code>	La clase y todas sus clases hijas	Datos que la hija necesita pero afuera no debe ver
<code>internal</code>	Solo dentro del mismo proyecto/ensamblado	Clases de utilidad internas al proyecto



Regla práctica: empieza siempre en `private`. Si la hija necesita verlo → `protected`. Si el exterior necesita usarlo → una

Propiedades con validación — código completo

```
class CuentaBancaria
{
    // — Campo privado con propiedad protegida
    

---


    private double _saldo;
    public double Saldo
    {
        get => _saldo;
        private set => _saldo = value >= 0 ? value
: 0;
    }

    public string Titular { get; private set; }

    // — Constructor con validación
    

---


    public CuentaBancaria(string titular, double
saldoInicial)
    {
        Titular = titular;
        Saldo = saldoInicial;
    }
}
```

Beneficios

- ✓ No puedes poner saldo negativo directamente
- ✓ La lógica de validación está en UN lugar
- ✓ Cambiar la implementación no afecta al usuario
- ✓ IntelliSense muestra solo la interfaz pública

04

Abstracción

Clases abstract e interfaces

¿Qué es la abstracción? — el concepto

💡 Abstraer = quedarse solo con lo esencial. Una clase abstract define QUÉ debe hacer una familia de objetos, sin decir CÓMO. Las clases concretas completan el 'cómo'. Nadie puede crear un objeto de una clase abstract directamente.

Clase Abstract

- Puede tener código implementado Y métodos abstract
- Puede tener campos y propiedades normales
- Se hereda con ':'
- Una clase solo puede heredar UNA abstract
- Ej: Figura, Empleado, Forma

Interface

- Solo tiene firmas de métodos (sin código)
- No tiene campos, solo propiedades abstractas
- Se implementa con ':'
- Una clase puede implementar VARIAS interfaces
- Ej: IComparable, IEnumerable, IDisposable

📌 Para esta semana nos enfocamos en abstract. Interfaces se ven a fondo en proyectos (semana 13+). La idea: ambas

Clase abstract — jerarquía Figura con área polimórfica

```
// — CLASE ABSTRACT: define el contrato —————  
abstract class Figura  
{  
    public string Color { get; set; }  
  
    protected Figura(string color) => Color = color;  
  
    // Método abstract: OBLIGA a las hijas a implementarlo  
    public abstract double Area();  
    public abstract double Perimetro();  
  
    // Método concreto: disponible para todas las hijas  
    public void MostrarInfo()  
        => Console.WriteLine($"{GetType().Name} ({Color}): Área={Area():F2},  
Perim={Perimetro():F2}");  
}  
  
// — CLASES CONCRETAS —————  
class Circulo : Figura
```



Demo en vivo — Jerarquía Figura

El docente construye: Figura abstract → Circulo → Rectangulo → Triangulo
Lista polimórfica que llama Area() para cada figura

Ejercicio explicado — Jerarquía

Figura polimórfica completa

```
abstract class Figura {  
    public string Nombre { get; }  
    protected Figura(string nombre) => Nombre = nombre;  
    public abstract double Area();  
    public abstract double Perimetro();  
    public virtual void Describir()  
        => Console.WriteLine($"{Nombre} → Área:{Area():F2} | Perim:{Perimetro():F2}");  
}  
  
class Circulo : Figura {  
    double R;  
    public Circulo(double r) : base("Círculo") => R = r;  
    public override double Area()          => Math.PI * R * R;  
    public override double Perimetro() => 2 * Math.PI * R;  
}  
  
class Rectangulo : Figura {  
    double B, H;  
    public Rectangulo(double b, double h) : base("Rectángulo") { B=b; H=h; }  
    public override double Area()          => B * H;  
    public override double Perimetro() => 2*(B+H);  
}
```



¡Tu turno! — Jerarquía Empleado/Gerente/Vendedor

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide

Jerarquía Empleado — Gerente y Vendedor



30 min

Clase base abstract Empleado

- Propiedades: Nombre, Cargo, SalarioBase (protected set)
- Constructor con parámetros para los 3 datos
- Método abstract: CalcularSueldo() → double
- Método concreto: MostrarInfo() → imprime todo con CalcularSueldo()
- Override ToString(): devuelve 'Cargo: Nombre | Sueldo: X'

Clase Gerente : Empleado

- Agrega: Bono (double, de solo lectura externa)
- Constructor: nombre, salario, bono — llama base()
- override CalcularSueldo(): SalarioBase + Bono
- override MostrarInfo(): llama base.MostrarInfo() y agrega el bono

Clase Vendedor : Empleado

- Agrega: Comision (double) y MetaVentas (double)
- Constructor: nombre, salario, comisión, meta
- override CalcularSueldo(): si Comision >= MetaVentas → salario+comisión; si no → solo salario
- Propiedad CumplioCuota → bool

Errores comunes — Semana 7

(Pilares POO)

❌ Olvidar override — el método no se sobrescribe

```
class Gerente : Empleado {  
    // ❌ sin override — es un método NUEVO, no  
    sobrescribe  
    public double CalcularSueldo() => SalarioBase +
```

❌ Sin override, el método oculta al padre pero no lo sobrescribe. El polimorfismo NO funciona.

```
class Gerente : Empleado {  
    public override double CalcularSueldo() // ✅  
        => SalarioBase + Bono;  
}
```

❌ Instanciar una clase abstract

```
abstract class Figura { public abstract double  
Area(); }  
// ...  
var f = new Figura(); // ❌ CS0144
```

❌ CS0144: Cannot create an instance of the abstract type 'Figura'

```
// ✅ Solo se instancian las clases concretas:  
var c = new Circulo(5);  
Figura f = new Circulo(5); // ✅ variable del tipo  
base
```

❌ Acceder a miembro protected desde fuera de la jerarquía

```
class Empleado { protected double SalarioBase; }  
class Otro { // ← NO hereda de Empleado  
    void Test(Empleado e) => e.SalarioBase = 0; //
```

❌ CS0122: 'Empleado.SalarioBase' is inaccessible due to its protection level

```
// ✅ Solo desde dentro de la clase o sus hijas:  
class Gerente : Empleado {  
    void Ajustar() => SalarioBase *= 1.1; // ✅  
}
```

Semana 7 — Los 4 Pilares de la POO

- ✓ **Herencia ':'** — La hija reutiliza el código del padre y puede extenderlo.
- ✓ **virtual / override** — Anunciar que un método se puede redefinir / hacerlo.
- ✓ **base()** — Llama al constructor o método del padre desde la hija.
- ✓ **protected** — Visible en la clase y sus hijas — no afuera.
- ✓ **Polimorfismo** — Un mismo mensaje actúa diferente según el tipo real del objeto.
- ✓ **Encapsulamiento** — private campos, public/protected propiedades, interfaz controlada.
- ✓ **abstract** — Define el QUÉ, obliga a implementar el CÓMO en las hijas.

TAREA

Para la próxima clase — Guía de ejercicios



E1

Fácil

Jerarquía Animal → Perro/Gato

E2

Fácil

Encapsular clase Producto

E3

Fácil

Clase abstract Vehiculo

E4

Medio

Jerarquía Figura con 4 subclases

E5

Medio

Polimorfismo con lista de objetos

E6

Medio

Jerarquía Cuenta bancaria

E7

Difícil

Sistema de Empleados completo

E8

Difícil

Tienda con jerarquía de Productos